조건문

조건문

조건에 따라 프로그램의 흐름을 분기해서 각각 다른 처리를 하도록 하는 명령문

if 문과 switch 문을 제공하고 있어 프로그램의 흐름을 제어할 수 있음

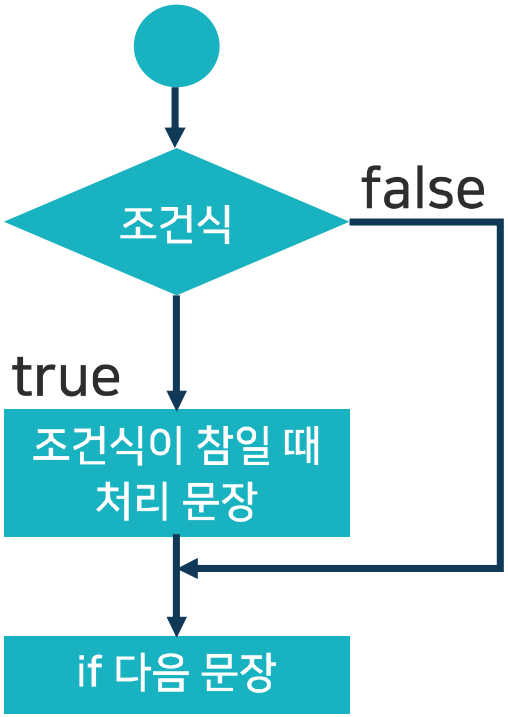
프로그램의 흐름	논리적 조건을 만들어서 프로그램 문장을 실행하는 것

조건문을 잘 다루기 위해서는 조건식을 잘 만들 수 있어야 함

if() 문

```
if (condition) {
    // 문장
}
```

- 괄호() 안의 condition은 값은 true 혹은 false 값을 가짐
- 이 논리값으로 조건문의 실행 여부를 판단
- 괄호 안은 일반적으로 관계 연산식이나 논리 연산식을 사용



참(true)일 경우

if 구문 바로 아래의 문장을 수행함

거짓(false)일 경우

if 구문 바로 아래의 문장은
실행되지 않고
if 구문이 종료됨

if() 문

if 문의 조건식이 참일 때 처리하는 문장은 하나 또는 여러 개의 문장 수도 있음

처리 문장이 한 라인일 경우 블록 기호({ })를 사용하지 않아도 됨

조건에 맞는 처리 문장이 여러 라인일 경우 블록 기호({ })를 반드시 사용해야 함

! 괄호를 사용하지 않는 경우 두 번째 문장은 조건의 결과로 실행하는 문장으로 인식하지 않음

if() 문

예시

시험 점수가 60점 이상일 때 “PASS”를 출력하는 프로그램을 만들기

점수(데이터) 값을 비교해서 결과를 출력하는 흐름을 만들어주어야 함

```
if (score >= 60) {
    System.out.println("PASS");
}
```

if() 문

예시

```
package chapter04;

public class IfTest {

    public static void main(String[] args) {
        int score = 91;

        if (score >= 60) {
            System.out.println("PASS"); // 조건식의 결과가 참일 때 “PASS”를 출력
        }
    }
}
```

if() 문

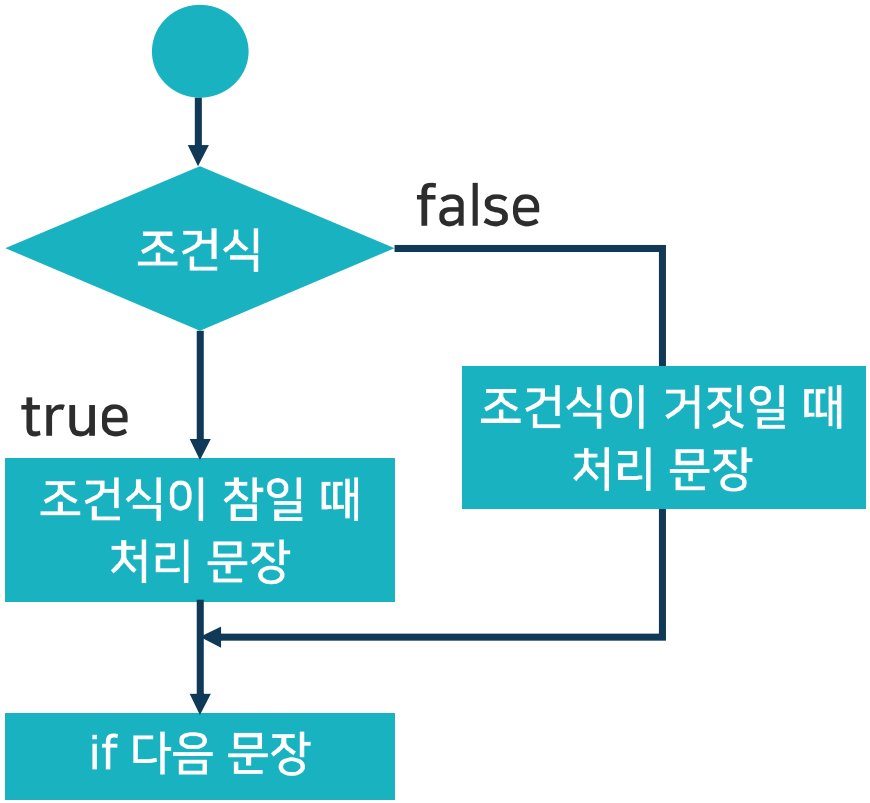
예시

```
package chapter04;

public class IfTest2 {

    public static void main(String[] args) {
        int age = 20;

        if (age > 18) {
            System.out.print("18세 이상입니다. 투표가 가능합니다.");
        }
    }
}
```

if() ~ else 문

프로그램에서 조건을 사용할 때 비교문의 결과에 따라 참일 때와 거짓일 때 각각 처리해야 할 흐름이 있다면 if()~else 구문을 사용

```
if (condition) {
    // 조건식이 true일 때
    // 실행할 문장
} else {
    // 조건식이 false일 때
    // 실행할 문장
}
```



if() ~ else 문

예시

점수가 60점 이상인 경우에는 “PASS”를 출력 하고, 60점 미만인 경우 “FAIL”을 출력하는 프로그램 만들기

```
if (score >= 60) {
    // 60점 이상일 때
    System.out.println("PASS");
} else {
    // 60점 이상이 아닐 때 즉, 60점 미만일 때
    System.out.println("FAIL");
}
```

if() ~ else 문

예시

```
package chapter04;

public class IfelseTest {

    public static void main(String[] args) {
        int score = 51;

        if (score >= 60) { // score 변수의 값을 60 이상인지를 비교
            // 60점 이상일 때
            System.out.println("PASS");
        } else { // score 변수의 값을 60 이상인지를 비교 값이 false 일 때
            // 60점 이상이 아닐 때 즉, 60점 미만일 때
            System.out.println("FAIL");
        }
    }
}
```

if() ~ else 문

예시

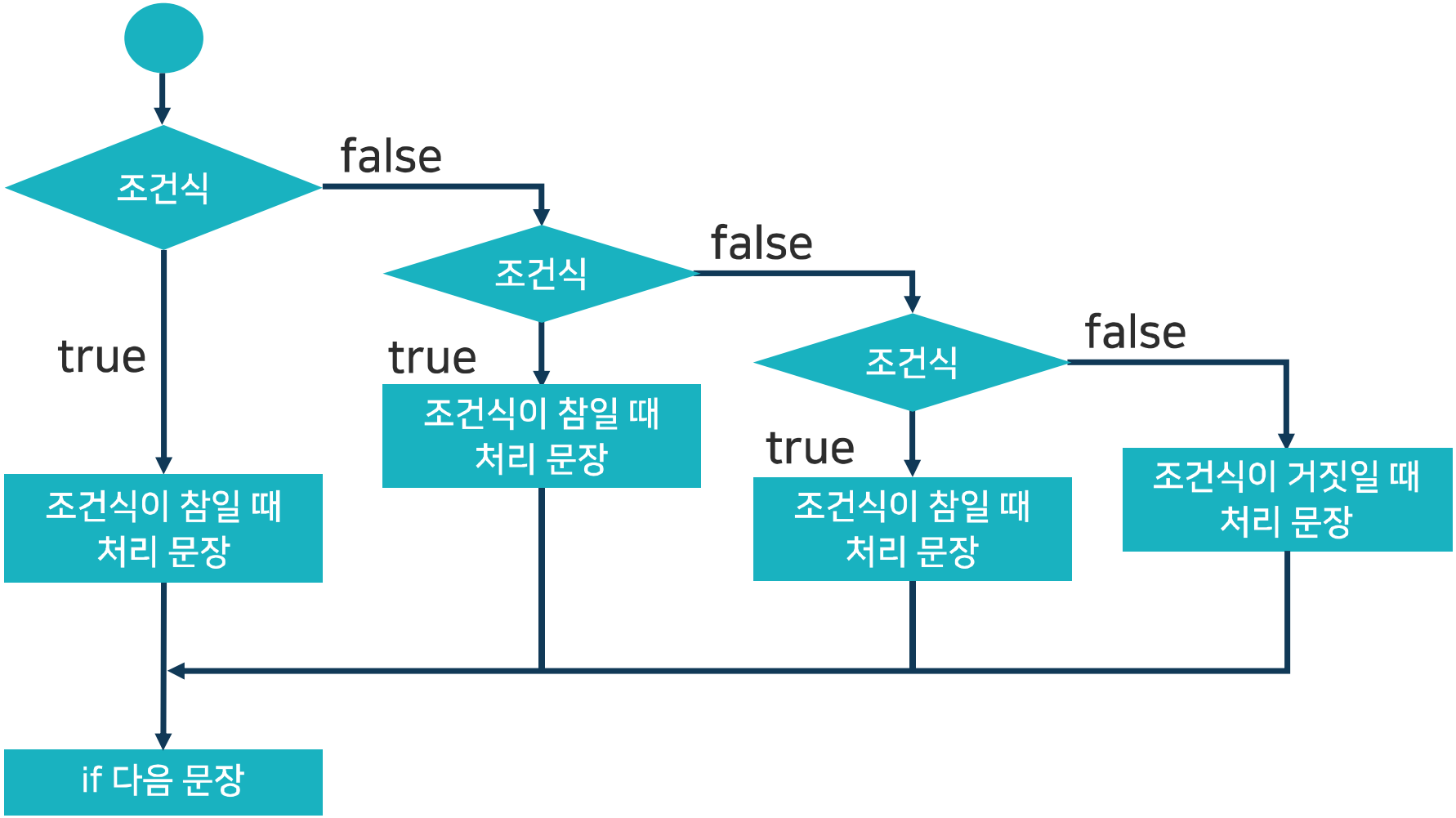
```
package chapter04;

public class IfelseTest2 {

    public static void main(String[] args) {
        int number = 21;

        if (number % 2 == 0) { // number 변수의 값을 2로 나눈 나머지가 0인지 비교
            System.out.println("짝수");
        } else { // 비교 값이 false 일 때
            System.out.println("홀수");
        }
    }
}
```

if() ~ else if() ~ else 문

여러 조건으로 판별할 때 다중 if ~ else if() else 구문을 사용

```
if (condition){
    // 문장1
} else if (condition){ // if (조건)의 결과가 false 이고, 이 조건을 비교
    // 문장2
} else { // if(조건)의 결과가 false이고, else if(조건)의 결과도 false
    // 문장3
}
```

if() ~ else if() ~ else 문



if() ~ else if() ~ else 문

예시

다중 if ~ elseif ~ else 구문을 사용하여 아래의 요구 조건에 맞게 프로그램 작성 예제

- 점수가 90점 이상인 경우 “A”를 출력
- 80점 이상이면서 90점 미만인 경우 “B”를 출력
- 앞의 두 조건에 맞지 않는 이외의 경우에는 “C”를 출력하는 프로그램

```
if (score >= 90) {
    // true □□90점 이상
    System.out.println("A");
} else if (score >= 80) {
    // true □□90점 이상이 아니고(90점 미만), 80점 이상인 경우
    System.out.println("B");
} else {
    // false □□90점 이상도 아니고, 80점 이상도 아닌 경우
    System.out.println("C");
}
```

if() ~ else if() ~ else 문

예시

```
package chapter04;

public class IfelseIfTest {

	public static void main(String[] args) {
		int score = 91;

		if (score >= 90) {
			// 90점 이상
			System.out.println("A");
		} else if (score >= 80) {
			// 90점 이상이 아니고, 80점 이상인 경우
			System.out.println("B");
```

if() ~ else if() ~ else 문

예시

```
                    } else {
                        // 90점 이상도 아니고, 80점 이상도 아닌 경우
                        System.out.println("C");
                    }
            }
}
```

if() ~ else if() ~ else 문

예시

```
package chapter04;
public class IfelseIfTest2 {
	public static void main(String[] args) {
		int number = -13;

		if (number > 0) { // number 변수의 값이 0보다 크면 “양수”라고 출력
			System.out.println("양수");
		} else if (number < 0) {  // number 변수의 값이 0보다 작으면 “음수”라고 출력
			System.out.println("음수");
		} else { // 위의 두 조건 모두 false 일 때 0 출력
			System.out.println("0");
		}
	}
}
```

조건식

예시

일반적인 문제	추출된 조건식
입력된 수가 양수인 경우	(input > 0)
두 과목 성적이 모두 80점 이상인 경우	(score1 >= 80 && score2 >= 80)
두 과목 성적 중에서 하나 이상이 80점 이상인 경우	(score1 >= 80 || score2 >= 80)
두 과목 성적의 합이 150인 경우	(score1 + score2 == 150)
입력된 두 수가 모두 짝수인 경우	(input1 % 2 == 0 && input2 % 2 ==0)
입력된 점수가 60보다 크고 100보다 작거나 같은 경우	(score > 60 && score <= 100)
직급이 7 또는 8급이고, 나이가 30대(30-39)인 경우	((grade == 7 grade == 8) && (30 <= age && age <= 39))

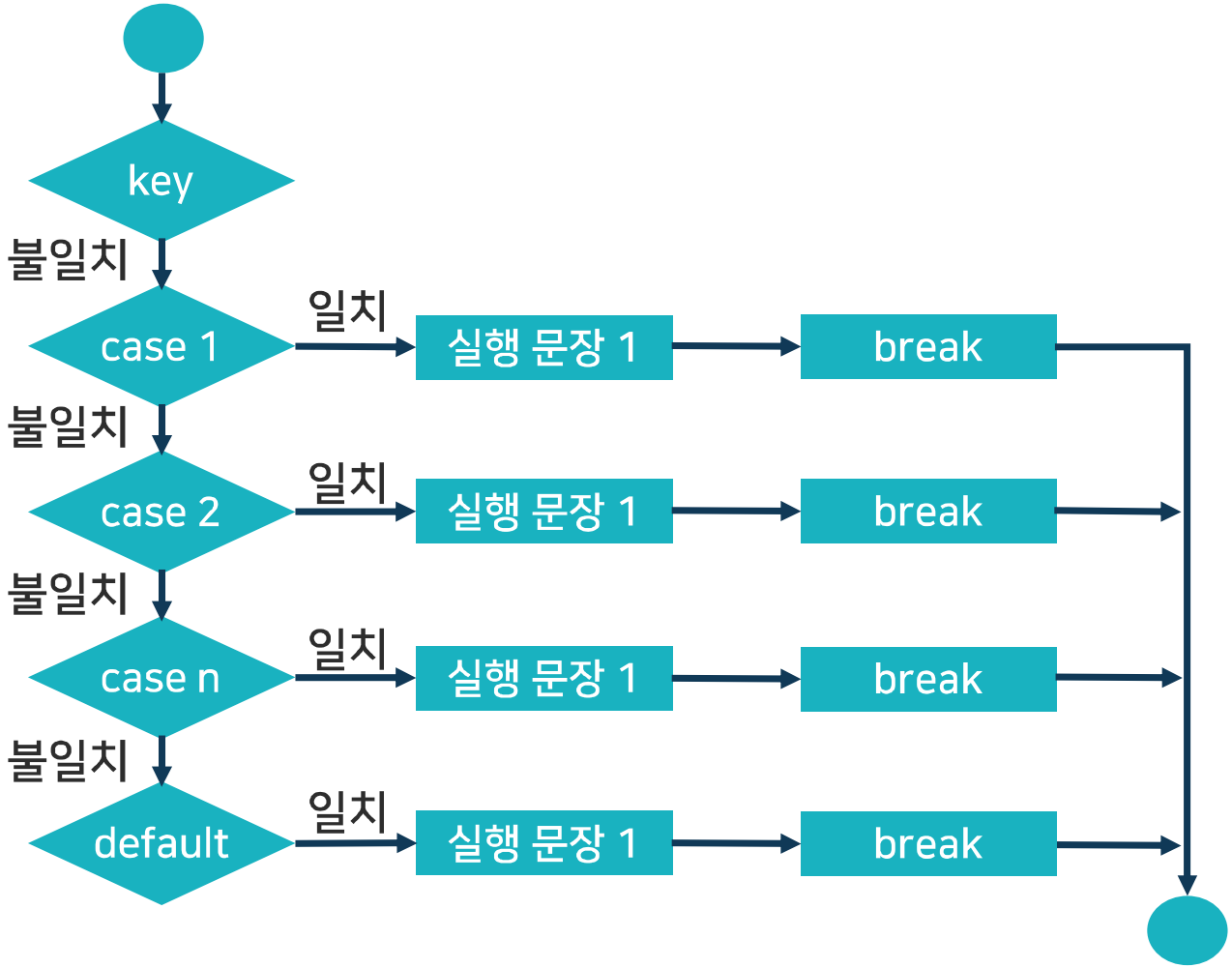
switch ~ case 문

switch ~ case 문은 여러 개의 조건이 필요할 때 사용

범위를 비교하는 연산보다는 하나의 특정 값을 비교하여 프로그램의 흐름을 만들 때 사용

```
switch (key){
    case value1:
        // key값과 value1의 값이 일치하는 경우 실행할 문장1
        break;
    case value2:
        // key값과 value2의 값이 일치하는 경우 실행할 문장1
        break;
    default:
        // key값과 value1, value2 모두 일치하지 않은 경우 실행할 문장1
        break;
}
```

switch ~ case 문



switch ~ case 문

예시

switch ~ case 구문을 사용하여 아래의 요구 조건에 맞게 프로그램 작성 예제

- 점수가 90점 이상인 경우 “A”를 출력
- 80점 이상이면서 90점 미만인 경우 “B”를 출력
- 앞의 두 조건에 맞지 않는 이외의 경우에는 “C”를 출력하는 프로그램

```
switch ( score/10 ){
    case 10 :
        System.out.println(“A”);
        break;
    case 9:
        System.out.println(“A”);
        break;
    case 8:
        System.out.println(“B”);
        break;
    default:
        System.out.println(“C”);
        break;
}
```

switch ~ case 문

예시

```
package chapter04;

public class SwitchCaseTest1 {

	public static void main(String[] args) {
		int score = 93;

		switch (score/10){ // socre 변수의 값을 10으로 나눈 몫을 비교하는 값으로 사용
		case 10 : // score/10 == 10,
			System.out.println("A");
			break;
		case 9: // score/10 == 9, 90이상
			System.out.println("A");
			break;
```

switch ~ case 문

예시

```
                case 8: // score/10 == 8, 80이상 90 미만
                        System.out.println("B");
                        break;
                default: // score/10!=10 and score/10 !=9 and score/10 !=8 , 80 미만
                        System.out.println("C");
                        break;
                }
        }
}
```

switch ~ case 문

break와 default의 생략

switch 구문에서 break와 default 문장은 생략 가능

- break문의 경우는 각 case 구문에서 value 값이 일치하는 위치에서 구문이 실행이 되고 아래 나오는 case 구문이 실행됨
- break 키워드가 없으면 아래에 있는 모든 case 구문이 실행됨

  ! key 값과 value1이 일치한다면 문장1, 문장2, 문장3이 모두 실행됨

```
switch ( key ){
    case value1:
        // 문장1
    case value2:
        // 문장2
    default:
        // 문장3
}
```

switch ~ case 문

break와 default의 생략

```
package chapter04;

public class SwitchCaseTest2 {

    public static void main(String[] args) {
        int score = 93;

        switch (score/10){
        case 10 :
            System.out.println("A");
        case 9: // 93/10 == 9 이기 때문에 case8 일 때와 default 까지 실행
            System.out.println("A");  // 출력됨
        case 8:
            System.out.println("B"); // 출력됨
```

switch ~ case 문

break와 default의 생략

```
        default:
            System.out.println("C"); // 출력됨
        }
    }
}
```

switch ~ case 문

break와 default의 생략

default 구문이 없을 때 case 문들의 조건에 맞지 않는 경우에는 아무것도 처리가 되지 않음

```
package chapter04;

public class SwitchCaseTest3 {

    public static void main(String[] args) {
        int score = 75;
        switch (score/10){
        case 10 :
            System.out.println("A");
            break;
        case 9:
            System.out.println("A");
            break;
        case 8:
            System.out.println("B");
            break;
        }
    }
}
```

조건문의 중첩

```
package chapter04;

public class IfNestedTest1 {

    public static void main(String[] args) {

        int num = 10;

        if(num>0) {
            System.out.println("num은 양의 정수 입니다.");
        } else {
            if(num<0) {
                System.out.println("num은 음의 정수입니다.");
            } else {
                System.out.println("num은 0 입니다.");
            } // if(num<0) 종료
        } // if(num>0) 종료
    }
}
```

반복문

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Java
C++

반복문

사용자(프로그래머)가 정의하는 횟수만큼 반복이 되도록 해주는 명령문

프로그램이 처리하는 대부분의 코드에 반복적인 형태가 많아 가장 많이 사용되는 제어문 중 하나

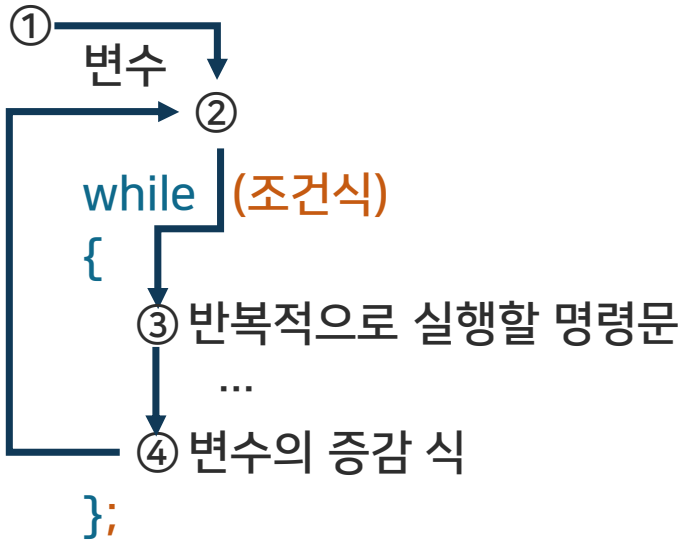
반복문의 종류	while 문, do ~ while 문, for 문

while 문

사용자(프로그래머)가 정의하는 횟수만큼 반복이 되도록 해주는 명령문

while 키워드 뒤에 오는 괄호() 안에 사용자가 조건을 정의하고 조건을 만족할 때
반복 블록({ }) 안의 명령 문장들을 실행

```
변수 // 반복의 횟수를 저장
while ( 조건식 ) {
    //조건식의 결과가 참일 때
    // 반복적으로 실행할 명령문
    ...
    // 변수의 증감 식
}
```



while 문

반드시 괄호 안의 값은 boolean 타입의 논리값이어야 함

반복 문을 탈출하기 위한 체크 변수를 반복문을 정의하기 전에 선언해 주어야 함

이 변수는 반복의 횟수를 저장하도록 하는 변수로 변수의 증감 연산이 반드시 정의되어서
이 변수의 값으로 반복의 구문이 탈출이 되도록 해주어야 함

while 문

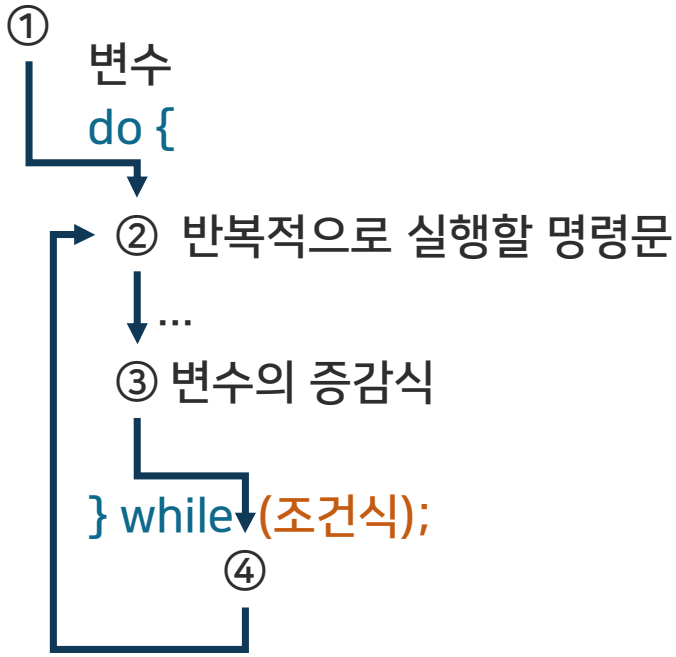
```java
package chapter04;

public class WhileLoop {

    public static void main(String[] args) {

        int num = 0; // 초기식, 반복을 위한 조건에 사용하는 변수

        while (num < 5) { // 반복해야 하는 구문
            System.out.println(num); // 반복해야 하는 구문
            num++; // num 의 값을 증가하는 것은 탈출을 위한 것입니다.
        }
    }
}
```

do while 문

조건식을 먼저 확인하는 것이 아니라 반복 처리할 블록{}을 먼저 실행한 후 반복의 조건을 확인함

조건과 상관없이 먼저 한번을 실행해야 하는 경우 do ~ while 반복문을 사용

```
변수 // 반복의 횟수를 저장
do {
    // 조건식의 결과가 참일 때
    //실행할 명령문들
} while (조건식);
```



do while 문

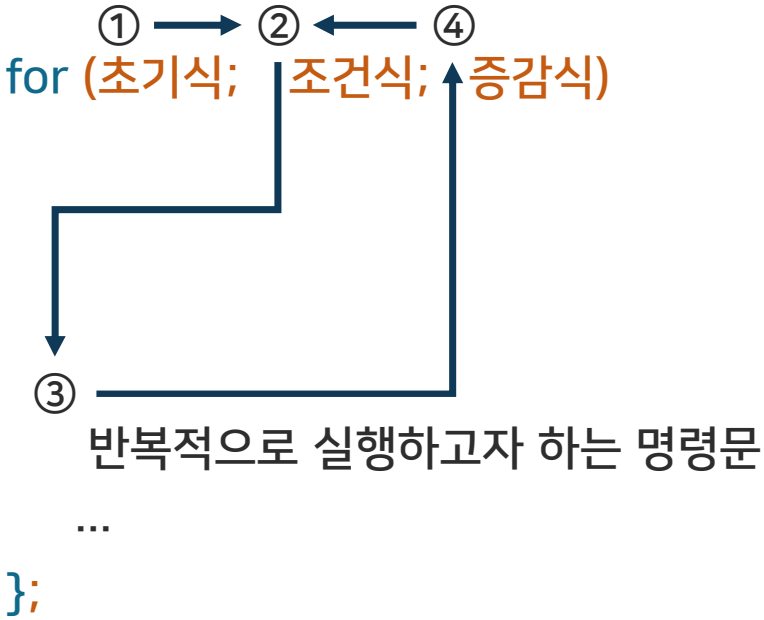
```java
package chapter04;

public class DoWhileLoop {

    public static void main(String[] args) {

        int num = 0; // 초기식, 반복을 위한 조건에 사용하는 변수

        do {
            System.out.println(num);
            num++;
        } while (num < 5); // 반복의 여부를 판별하는 조건식
    }
}
```

for 문

초기식, 조건식, 증감식을 반복문의 정의부에서 정의해 놓고 실행하는 반복문으로
반복의 횟수가 정해져 있을 때 주로 사용

배열이나 컬렉션의 리스트를 사용할 때 사용

```
for ( 초기식 ; 조건식 ; 증감식 )
{
    // 조건식의 결과가 참일 때
    //실행할 명령문
}
```



for 문

```
package chapter04;

public class ForLoop {

	public static void main(String[] args) {

		for (int i = 0; i < 5; i++) { // for 반복문을 정의 0~4까지 반복
			System.out.println(i); // 초기식의 변수 i를 출력
		}
	}
}
```

반복문의 중첩

반복문도 조건식과 마찬가지로 처리 블록{}안에 또 다른 반복문을 정의해서 처리할 수 있음

구구단의 경우 각 단이 증가하면서 각 단에서 2~9를 곱하는 패턴을 가지고 있음

2 × 1 = 2	3 × 1 = 3	4 × 1 = 4	5 × 1 = 5	6 × 1 = 6	7 × 1 = 7	8 × 1 = 8	9 × 1 = 9
2 × 2 = 4	3 × 2 = 6	4 × 2 = 8	5 × 2 = 10	6 × 2 = 12	7 × 2 = 14	8 × 2 = 16	9 × 2 = 18
2 × 3 = 6	3 × 3 = 9	4 × 3 = 12	5 × 3 = 15	6 × 3 = 18	7 × 3 = 21	8 × 3 = 24	9 × 3 = 27
2 × 4 = 8	3 × 4 = 12	4 × 4 = 16	5 × 4 = 20	6 × 4 = 24	7 × 4 = 28	8 × 4 = 32	9 × 4 = 36
2 × 5 = 10	3 × 5 = 15	4 × 5 = 20	5 × 5 = 25	6 × 5 = 30	7 × 5 = 35	8 × 5 = 40	9 × 5 = 45
2 × 6 = 12	3 × 6 = 18	4 × 6 = 24	5 × 6 = 30	6 × 6 = 36	7 × 6 = 42	8 × 6 = 48	9 × 6 = 54
2 × 7 = 14	3 × 7 = 21	4 × 7 = 28	5 × 7 = 35	6 × 7 = 42	7 × 7 = 49	8 × 7 = 56	9 × 7 = 63
2 × 8 = 16	3 × 8 = 24	4 × 8 = 32	5 × 8 = 40	6 × 8 = 48	7 × 8 = 56	8 × 8 = 64	9 × 8 = 72
2 × 9 = 18	3 × 9 = 27	4 × 9 = 36	5 × 9 = 45	6 × 9 = 54	7 × 9 = 63	8 × 9 = 72	9 × 9 = 81

반복문의 중첩

```
package chapter04;

public class DuplicatedWhileLoop {

    public static void main(String[] args) {

        int i=1; // 출력할 때의 행을 표현하기 위한 반복문의 조건을 만들기 위한 변수

        while(i<10) { // 반복의 조건 : 1~9

            int j=2; // 단을 표현하기 위한 변수 : 2~9

            while(j<10) {
                System.out.print(j + " x" + i + " = " + j*i + "₩t");
                j++; // j변수에 1을 증가시킵니다. 탈출을 위한 증가식입니다
            }
            System.out.println();
            i++; // 행의 반복을 위한 증가식, 탈출을 위한 증가식
        }
    }
}
```

반복문의 중첩

```
package chapter04;

public class DuplicatedForLoop {

    public static void main(String[] args) {

        for (int i = 1; i < 10; i++) { // 수직방향(행)의 반복을 위한 for 반복문을 정의

            for (int j = 2; j < 10; j++) { //수평방향(열)의 반복을 위한 for 반복문을 정의
                System.out.print(j + " x" + i + " = " + j*i + "₩t");
            }

            System.out.println();
        }
    }
}
```

반복제어를 위한 키워드

break

break 키워드는 반복문 내에서 사용함

반복문 안에서 break 키워드를 만나면 조건식과 관계없이 바로 반복문을 종료 시키고
반복문 다음 문장을 실행함

반복의 횟수를 모르지만 반복 문을 사용해야 하는 경우

반복 블록{} 내에 조건을 두고 조건에 충족하면 반복 문을 벗어나도록 할 때 사용

→ 반복문 내에서 조건식의 판단 결과와 상관없이 반복문을 완전히 빠져나가고 싶을 때 사용

반복제어를 위한 키워드

break

```
package chapter04;

public class WhileLoopBreak {

    public static void main(String[] args) {

        int num = 0; // 초기식, 반복을 위한 조건에 사용하는 변수

        while (true) { // 무한 반복을 하기 위한 조건 : true

            if(num==5) {
                break; // 반복문을 벗어남
            }

            System.out.println(num);

            num++; // 탈출을 위한 증가식
        }
    }
}
```

반복제어를 위한 키워드

continue

continue 키워드는 반복문 내에서 사용함

반복문 내에서 continue 키워드를 만나면 continue 아래 문장들은 건너뛰고 블록의 마지막으로 이동해서 다시 조건식으로 이동함

반복문 내에서 특정 조건일 때 처리하지 않고 건너뛰게 만들 때 사용함

반복제어를 위한 키워드

continue

```
package chapter04;

public class WhileLoopContinue {

    public static void main(String[] args) {

        int num = 0; // 초기식, 반복을 위한 조건에 사용하는 변수

        while (++num < 10) { // num의 값이 10보다 작은 조건일 때 반복

            if(num%2 == 1 ) { //홀수 인지를 판별하는 조건문
                continue; // 조건식으로 이동
            }

            System.out.println(num);
        }
    }
}
```